

Fashion Trend Intelligence Pipeline Improvement Roadmap

From Static MVP to Continuously Learning System

Phase 1 — Phase 2 — Phase 3

By: Pratik Dongre

Built with assistance from Claude

Table of Contents

1. Executive Summary
2. Current Architecture Assessment
3. Target Architecture
4. Phase 1: Live Data Pipeline
5. Phase 2: Feedback Loop & Backtesting
6. Phase 3: MLOps Infrastructure
7. Expected Impact
8. Risk Assessment

1. Executive Summary

The current Fashion Trend Intelligence Pipeline is a strong static MVP that demonstrates the value of merging social signals with sell-through data. However, it has three fundamental limitations that prevent it from delivering sustained value:

Limitation	Current State	Target State
Data Freshness	Static CSVs	Live weekly data from Instagram, retailer catalogs, Reddit, Google Trends
Learning Ability	One-shot analysis; no feedback	Walk-forward backtesting, accuracy measurement, automatic weight optimization
Operational Maturity	Manual notebook execution	Automated weekly pipeline with drift monitoring, experiment tracking, and scheduled retraining

This roadmap describes three phases that progressively address each limitation, transforming the static prototype into a system that gets smarter with every iteration.

2. Current Architecture Assessment

What Works Well

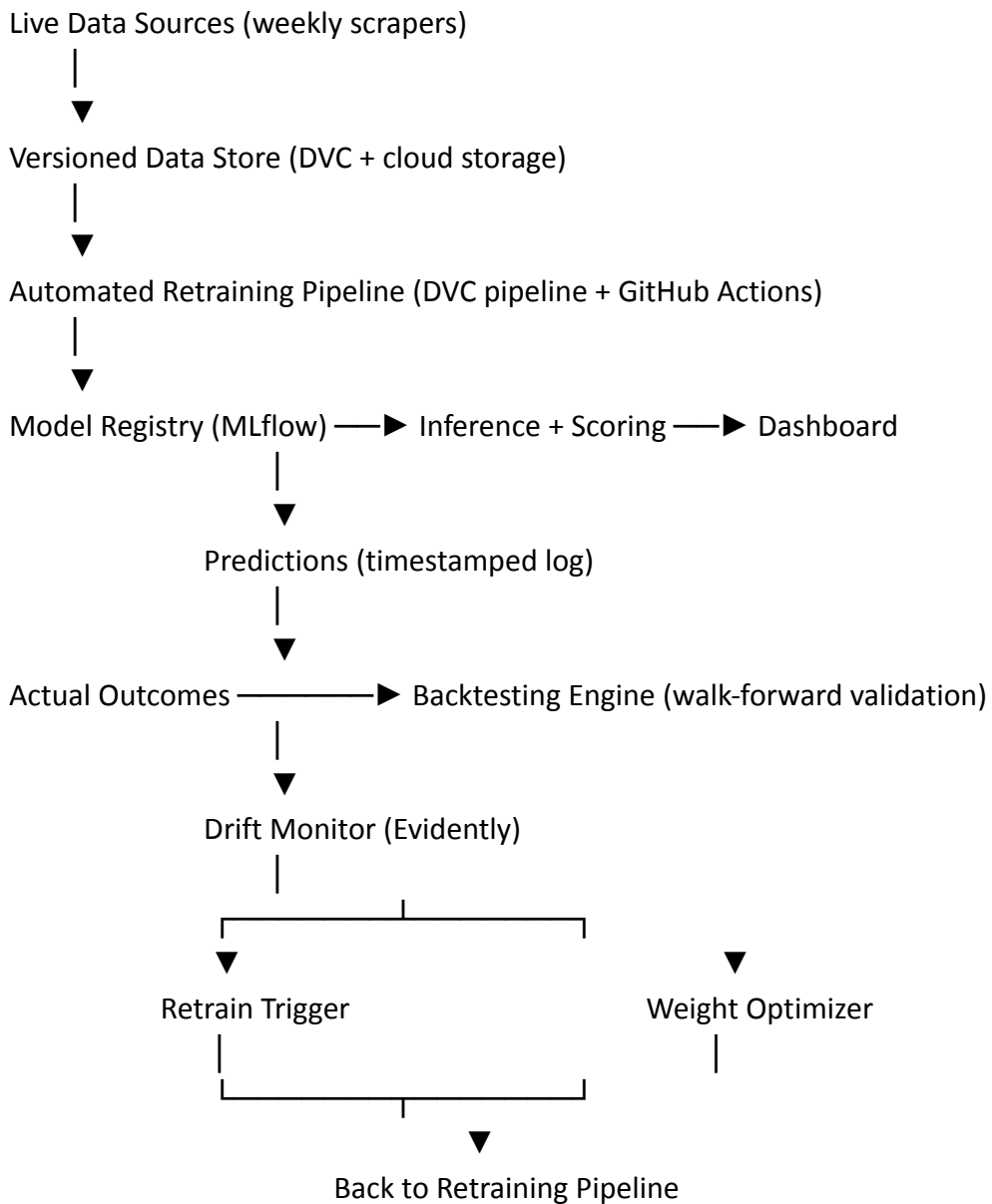
- Modular src/ package with clean separation of concerns
- Three complementary ML models providing rich social signals
- Configuration-driven design (all parameters in config.py)
- Five signal composite scoring with transparent methodology
- Quadrant classification that maps directly to business actions
- Interactive Plotly dashboard for stakeholder consumption

What's Missing

- No live data ingestion. Everything is static
- No feedback loop, cannot measure if predictions were correct
- No model monitoring, cannot detect when models degrade
- No automated execution, requires manual notebook runs
- No experiment tracking, cannot compare model versions over time
- No prediction history, cannot trend accuracy over time
- Scoring weights are manually set, not data-driven

3. Target Architecture

The target architecture introduces a continuous feedback loop:



The key difference: every prediction is logged, compared against actual outcomes, and used to improve the next cycle. The system gets smarter over time.

4. Phase 1: Live Data Pipeline

Replace static CSVs with live, scheduled data ingestion from four sources.

4.1 Instagram Hashtag Scraping

Attribute	Detail
Tool	Apify (free tier for testing, ~\$0.50/1K results at scale)
Hashtags	#streetwear, #quietluxury, #y2kfashion, #minimalistfashion, #bohostyle, #athleisure, #sustainablefashion, #oversized, #boldprints, #preppystyle etc etc
Data Collected	Caption text, likes, comments count, timestamp, media type, hashtags
Schedule	Weekly automated scrape
New Module	src/instagram_signals.py
NLP Processing	Run the same sentiment + zero-shot + embedding pipelines on Instagram captions
Key Advantage	Real social media signal instead of e-commerce reviews as proxy

4.2 Fashion Retailer Catalog Scraping

Attribute	Detail
Tool	Apify pre-built Actors (H&M, Zara) or custom Playwright scraper, In-House Data from future clients
Retailers	H&M new arrivals, Zara new arrivals, ASOS trending
Data Collected	Product name, category, color, price, 'new' flag, description, stock status
Schedule	Weekly automated scrape
New Module	src/catalog_scraper.py
Key Advantage	Shows what brands are actually producing NOW real-time supply signal

4.3 Reddit Fashion Subreddits

Attribute	Detail
Tool	PRAW (Python Reddit API Wrapper): free
Subreddits	r/streetwear, r/malefashionadvice, r/femalefashionadvice, r/fashion.... Indian fashion subreddits.
Data Collected	Post title, body text, upvotes, comments, timestamp
Schedule	Weekly top posts
New Module	src/reddit_signals.py
Key Advantage	Organic consumer fashion discussions, less marketing bias than Instagram

4.4 Automated Google Trends

The existing `src/google_trends.py` is modified to APPEND to a historical time series (`data/processed/trends_history.parquet`) instead of overwriting. This builds a rolling window of trend data that improves momentum calculations over time.

4.5 Data Pipeline Orchestrator

A new `src/data_pipeline.py` module orchestrates all data ingestion. It runs weekly/optimum cadence, pulls from all four sources, validates data quality, and saves versioned snapshots.

5. Phase 2: Feedback Loop & Backtesting

This is the most important upgrade. Without measuring prediction accuracy, the system can never learn or improve.

5.1 Walk-Forward Backtesting Framework

New module: `src/backtester.py`

The concept: use historical data to simulate the pipeline running at past dates, then compare predictions against what actually happened.

Example walk-forward cycle:

Weeks 1-8: Train on this data (features + sell-through)

Week 9: Pipeline predicts 'cargo pants = HOT, slip dress = EMERGING'

Weeks 10-12: Check actual sales, were cargo pants actually HOT?

Slide window forward by 1 week, repeat.

This produces a time series of prediction accuracy that shows whether the system is improving.

Backtesting Metrics

Metric	Definition	Target
Quadrant Accuracy	% of categories placed in the correct quadrant	$\geq 60\%$
Rank Correlation	Spearman correlation: predicted scores vs actual sales	≥ 0.5
Precision@K	Of top-K 'over-index' picks, how many actually outperformed?	$\geq 70\%$
Lead Time	How many weeks before a trend peaks does the system detect it?	≥ 3 weeks ? Maybe 4 weeks ? to give brand ample time

5.2 Prediction Logging

Every pipeline run logs its predictions with timestamps to `predictions_log.parquet` (append-only). When actual sales data arrives later, a join on (category, date_window) computes accuracy. This creates an auditable history of every prediction the system made.

5.3 Scoring Weight Auto-Tuning

The current scoring weights (25/25/20/15/15) are manually set. With backtesting data, we auto-optimize them using Optuna (Bayesian optimization).

The optimizer finds weights that maximize Precision@K and rank correlation against backtesting ground truth. Optimal weights may differ by season (Spring vs Winter), and the system stores per-season weight profiles.

New module: `src/weight_optimizer.py`

This is how the system LEARNS which signals matter most.

6. Phase 3: MLOps Infrastructure

Phase 3 automates everything built in Phases 1 and 2, so the system runs without manual intervention.

6.1 Experiment Tracking with MLflow

Every model training and scoring run is wrapped in MLflow:

- Log all hyperparameters, metrics, and model artifacts
- Compare retraining runs against previous versions
- Model Registry: promote best model to 'Production' stage
- Track whether each retraining actually improved performance

This creates a complete audit trail of model evolution.

6.2 Data Drift Monitoring with Evidently

New module: `src/drift_monitor.py`

Monitors two types of drift using the Evidently library:

- **Data drift:** Are incoming reviews/social posts changing in character? (new slang, new topics, language shifts)

- **Prediction drift:** Are the model's output distributions shifting unexpectedly?

When drift is detected, the system automatically triggers retraining. This prevents silent model degradation.

6.3 Automated Pipeline with DVC + GitHub Actions

The full pipeline is defined as a DVC DAG (`dvc.yaml`) with stages: `ingest` → `sell_through` → `social_signals` → `fusion` → `backtest`.

A GitHub Action (`.github/workflows/retrain.yml`) runs this weekly on a cron schedule. DVC only reruns stages whose dependencies changed, making execution efficient.

DVC also versions all data and model artifacts with lightweight pointer files in Git, while actual data resides in cloud storage (S3, GCS, or Google Drive).

6.4 Incremental Model Updates

Instead of full retraining every week, we will use incremental approaches:

- **Sentiment model:** Fine-tune LoRA adapter for 1 additional epoch on new labeled data
- **Embeddings + clusters:** Re-run HDBSCAN on the growing corpus clusters naturally evolve
- **Trend scorer weights:** Re-optimize via Optuna after each backtesting cycle
- **Time-series components:** Use `MLForecast.update()` for incremental updates without full retrain

7. Expected Impact

Capability	Current (MVP)	After Phase 1	After Phase 2	After Phase 3
Data Freshness	Static	Weekly/ regular live updates	Same + historical archive	Same + versioned (DVC)
Learning Ability	None (one-shot)	None	Backtesting + weight tuning	Auto-retrain on drift
Prediction Accuracy	Unvalidated	Unvalidated	Measured + improving	Continuously monitored
Model Maintenance	Manual	Manual	Manual + metrics	Fully automated
Social Signal Quality	Review text (proxy)	Real Instagram + Reddit	Same + feedback	Same + drift-corrected
Operational Effort	Manual notebook runs	Scripted ingestion	Same + logging	Zero-touch weekly

8. Risk Assessment

Risk	Phase	Likelihood	Impact	Mitigation
Instagram scraping blocked / rate-limited	1	Medium	High	Use Apify's proxy rotation; fall back to Reddit + Google Trends if blocked
Retailer website structure changes	1	Medium	Medium	Use Apify pre-built Actors (maintained by community); add alerting on scrape failures
Backtesting shows low accuracy	2	Low-Medium	Low	This is valuable information, it guides weight tuning and signal selection
Model drift goes undetected	3	Low (after Evidently)	High	Evidently runs on every pipeline execution; alerts trigger on threshold breach
GitHub Actions runner limits	3	Low	Medium	Free tier allows 2,000 minutes/month
Data quality degradation from live sources	1-3	Medium	Medium	Data validation checks in data_pipeline.py; schema enforcement; row count thresholds